DIAPOSITIVA 1: PORTADA

Motor V8 de JavaScript y Gestión de Datos en Node.js

Asignatura: Programación Web

Tema: Funcionamiento del motor V8, Call Stack, Memory Heap y análisis de requerimientos de un sistema en Node.js.

Integrantes:

- Nombre 1
- Nombre 2
- Nombre 3

Exposición

"Buenos días. En esta exposición hablaremos sobre el motor V8 de JavaScript, utilizado por Node.js y Google Chrome. Explicaremos cómo administra la memoria mediante el Call Stack y el Memory Heap, y presentaremos un análisis de requerimientos para un sistema desarrollado en Node.js."

DIAPOSITIVA 2: ¿QUÉ ES EL MOTOR V8?

Definición

V8 es un motor de JavaScript desarrollado por Google.

Su función principal es ejecutar código JavaScript de manera rápida y eficiente.

Es utilizado en:

- Google Chrome
- Node.js
- Microsoft Edge (basado en Chromium)

Características

- Compila JavaScript a código máquina.
- Optimiza el rendimiento.
- Gestiona automáticamente la memoria.

- Utiliza técnicas de optimización Just-In-Time (JIT).

Exposición

"El motor V8 fue desarrollado por Google para ejecutar JavaScript de forma eficiente. Su principal ventaja es que convierte el código JavaScript directamente en código máquina, permitiendo una ejecución mucho más rápida que otros métodos tradicionales."

DIAPOSITIVA 3: GESTIÓN DE MEMORIA EN V8

¿Cómo administra la memoria?

V8 divide la memoria principalmente en dos áreas:

Call Stack (Pila)

Almacena:

- Variables primitivas.
- Información de funciones.
- Direcciones de retorno.

Memory Heap (Montón)

Almacena:

- Objetos.
- Arreglos.
- Clases.
- Funciones complejas.

Exposición

"El motor V8 organiza la memoria en dos grandes estructuras. La primera es la pila o Call Stack, donde se guardan los datos simples y el control de ejecución. La segunda es el Heap, donde se almacenan estructuras más complejas como objetos y arreglos."

DIAPOSITIVA 4: CALL STACK (PILA)

Características

- Memoria rápida.
- Tamaño limitado.
- Organización LIFO (Último en entrar, primero en salir).
- Almacena tipos primitivos.

Tipos Primitivos

- Number
- String
- Boolean
- Undefined
- Null
- BigInt
- Symbol

Ejemplo

let edad = 20;

let nombre = "Carlos";

let activo = true;

Exposición

"Los tipos primitivos son datos simples que ocupan poco espacio. Por esta razón son almacenados directamente en el Call Stack. El acceso a estos datos es muy rápido y eficiente."

DIAPOSITIVA 5: MEMORY HEAP (MONTÓN)

Características

- Mayor capacidad de almacenamiento.
- Maneja estructuras dinámicas.
- Utiliza referencias de memoria.
- Permite almacenar grandes cantidades de información.

Ejemplo

```
let estudiante = {
  nombre: "Carlos",
  edad: 20
};


let notas = [8, 9, 10];
```

Exposición

"Cuando trabajamos con objetos o arreglos, estos se almacenan en el Heap porque requieren más espacio. Las variables guardadas en el Stack contienen solamente una referencia hacia la ubicación real de los datos en el Heap."

DIAPOSITIVA 6: DIFERENCIAS ENTRE STACK Y HEAP

Call Stack	**Memory Heap**
Almacena datos primitivos	Almacena objetos y arreglos
Más rápido	Más flexible
Menor capacidad	Mayor capacidad
Acceso directo	Acceso mediante referencias
Organización LIFO	Memoria dinámica

Importancia

Una mala gestión del Heap puede generar:

- Consumo excesivo de memoria.
- Lentitud en la aplicación.
- Fugas de memoria.

Exposición

"La diferencia principal es que el Stack almacena información simple y temporal, mientras que el Heap almacena estructuras complejas. Por eso es importante gestionar correctamente los objetos para evitar problemas de rendimiento."

DIAPOSITIVA 7: ANÁLISIS DE REQUERIMIENTOS DEL PROYECTO ABP

Problema

Se requiere desarrollar un sistema que permita gestionar datos estáticos y dinámicos utilizando Node.js.

Necesidades identificadas

- Registro de información.
- Consulta de registros.
- Modificación de datos.
- Eliminación de información.
- Almacenamiento eficiente.
- Uso adecuado de memoria.

Usuarios

- Administrador.
- Operadores del sistema.

Exposición

"Analizamos la necesidad de crear un sistema capaz de administrar información de manera eficiente. El sistema debe permitir registrar, consultar, actualizar y eliminar datos manteniendo un buen rendimiento."

DIAPOSITIVA 8: REQUERIMIENTOS FUNCIONALES

RF1

Registrar nuevos usuarios.

RF2

Consultar información almacenada.

RF3

Actualizar registros existentes.

RF4

Eliminar registros.

RF5

Gestionar datos dinámicos mediante objetos y arreglos.

RF6

Mostrar información organizada.

Exposición

"Los requerimientos funcionales representan las acciones que el sistema debe realizar para satisfacer las necesidades de los usuarios."

DIAPOSITIVA 9: REQUERIMIENTOS NO FUNCIONALES

Rendimiento

El sistema debe responder en menos de 2 segundos.

Uso de Memoria

Debe optimizar el uso del Stack y Heap.

Escalabilidad

Debe soportar crecimiento de usuarios y datos.

Seguridad

Debe proteger la información almacenada.

Mantenibilidad

El código debe ser fácil de modificar y actualizar.

Exposición

"Los requerimientos no funcionales establecen las características de calidad del sistema, como velocidad, seguridad, escalabilidad y uso eficiente de memoria."

DIAPOSITIVA 10: ESTRUCTURAS DE DATOS UTILIZADAS

Datos Estáticos

Variables primitivas:

```
let nombre = "Juan";

let edad = 20;
```

Datos Dinámicos

Objetos:

```
let usuario = {
 nombre:"Juan",
 edad:20
};
```

Arreglos:

```
let estudiantes = [];
```

Exposición

"Las variables primitivas se almacenan en el Stack, mientras que los objetos y arreglos se almacenan en el Heap. Esta organización permite gestionar eficientemente la memoria."

DIAPOSITIVA 11: CONCLUSIONES

Conclusión 1

El motor V8 optimiza la ejecución de JavaScript mediante compilación eficiente.

Conclusión 2

El Call Stack almacena datos simples y controla la ejecución de funciones.

Conclusión 3

El Memory Heap almacena estructuras complejas como objetos y arreglos.

Conclusión 4

Comprender la administración de memoria permite desarrollar aplicaciones más eficientes en Node.js.

Conclusión 5

La especificación de requerimientos facilita el desarrollo y mantenimiento del software.

Exposición

"Podemos concluir que el conocimiento del funcionamiento interno del motor V8 es fundamental para desarrollar aplicaciones eficientes. Además, una correcta definición de requerimientos permite construir sistemas de mejor calidad."

DIAPOSITIVA 12: GRACIAS

Gracias por su atención

Preguntas

"Muchas gracias por su atención. Estamos atentos a sus preguntas y comentarios."